

MODULE 3 — Synchronization (Highest Priority)

Senior SWE Interview Track — Operating Systems

3.1 Race Condition

1. Why Interviewers Ask This

The single most common OS/concurrency screen. Interviewers check that you can explain *why* `count++` is broken at the instruction level, not just recite a definition.

2. Core Concept

A race condition occurs when the result depends on the uncontrolled interleaving of concurrent accesses to shared state, with at least one write. A **data race** is the narrower memory-model term: two unsynchronized accesses, same location, one is a write — undefined behavior in C/C++/Go.

3. Internal Working

`count++` compiles to load → add → store. Two threads can both load the same value, both add, both store → one increment lost. Beyond interleaving: compilers reorder/cache values in registers, and CPUs reorder memory operations (store buffers) — so races break even on a single “logical” interleaving model. Correctness requires mutual exclusion or atomic read-modify-write plus proper memory ordering.

4. ASCII Diagram

```
count = 5
T1: load r=5      |
                  | T2: load r=5
T1: r=6, store 6  |
                  | T2: r=6, store 6
Result: 6 (should be 7) -> LOST UPDATE

Also: check-then-act race (TOCTOU):
T1: if (map.get(k)==null)  T2: if (map.get(k)==null)
T1: map.put(k, v1)         T2: map.put(k, v2)    -> both "won"
```

5. Real Production Example

- Double-spend / double-booking bugs: two requests read `balance=100`, both withdraw 100.
- Therac-25 radiation machine (race → lethal overdoses) — canonical story.
- TOCTOU security bugs: `access()` then `open()` on a path an attacker swaps in between.

6. Advantages

(N/A — a defect.) The relevant “advantage” is knowing benign-looking patterns that are actually racy: lazy init, check-then-act, read-modify-write.

7. Trade-offs

Fixes trade throughput for correctness: coarse locks (simple, contended), fine-grained locks (fast, deadlock-prone), atomics (fast, hard), immutability/message passing (clean, allocation cost).

8. Common Mistakes

- “It works in my tests” — races are probabilistic and load-dependent.
- Believing `volatile` (Java) fixes read-modify-write (it fixes visibility, not atomicity).
- Fixing the write but leaving unsynchronized reads (still a data race; may see torn/stale values).

9. Performance Implications

Races themselves are “fast”; every fix costs: uncontended atomic ~10–20 ns wasn’t free, contended cache-line ping-pong can be 100×. Design to *reduce sharing* first.

10. Common Interview Questions

- “Why is `i++` not thread-safe? Show the interleaving.”
- “Race condition vs data race?” “What is check-then-act / TOCTOU?”

11. Follow-up Questions

- “How would you find races in a large codebase?” (TSan, Go `-race`, code review of shared mutable state)
- “Can a race exist even if every individual operation is atomic?” (yes — atomicity of the *composite* is what matters: check-then-act)

12. Coding/Debugging Scenarios

- Counter off by ~0.1% only under load → lost updates; fix with atomic increment.
- Singleton occasionally constructed twice → unsynchronized lazy init; fix with once-initialization (`std::call_once`, `sync.Once`, holder idiom).

13. Best Practices

Minimize shared mutable state; prefer immutability and message passing; make composite operations atomic (one lock per invariant); run TSan/ `-race` in CI.

14. Practice Questions

1. Find the race: `if (!cache.contains(k)) cache.put(k, compute(k));`
2. Two services decrement inventory in a DB without transactions — describe the failure and three fixes (transaction, atomic UPDATE, optimistic version check).

3.2 Critical Section

1. Why Interviewers Ask This

Frames every locking discussion: interviewers want the three correctness requirements and whether you keep critical sections *small*.

2. Core Concept

A critical section is code that accesses shared state and must not be executed by more than one thread at a time. A correct solution needs: 1. **Mutual exclusion** — at most one thread inside. 2. **Progress** — if empty, someone waiting can enter (no needless blocking). 3. **Bounded waiting** — no thread waits forever (fairness/starvation bound).

3. Internal Working

Enforced by hardware primitives (atomic RMW: **LOCK**-prefixed instructions, LL/SC) wrapped by locks. Entry/exit protocol: acquire → critical section → release. Historical software-only solutions (Peterson’s algorithm) matter only to show why hardware atomics are needed on modern out-of-order CPUs (they break without memory barriers).

4. ASCII Diagram

```
T1: ---[acquire]===CS===[release]-----
T2: -----[acquire: BLOCKED].....[acquire]===CS===[release]--
Rule of thumb: shrink === (the CS) — do I/O, allocation,
logging, RPC *outside* the lock.
```

5. Real Production Example

Checkout flow: `reserve inventory + create order` is one logical critical section — implemented as a DB transaction (locks) or a distributed lock. The Big Kernel Lock removal from Linux is the classic “coarse CS → fine CS” scalability story.

6. Advantages

Restores sequential reasoning: invariants hold at CS boundaries.

7. Trade-offs

Serialization: by Amdahl’s law, a CS that is 5% of execution caps speedup at 20× regardless of cores.

8. Common Mistakes

- Doing blocking I/O or RPC inside a lock (an interview red flag and a prod outage pattern).
- Protecting *code* instead of *data* — the lock must be associated with the invariant/data it guards.
- Two code paths touching the same data, only one takes the lock.

9. Performance Implications

Lock hold time × acquisition rate = utilization of the “lock server”. Hold 1 ms at 2000 acq/s → the lock is 200% utilized → unbounded queueing. Measure hold times.

10. Common Interview Questions

- “What are the three requirements of a critical-section solution?”
- “How large should a critical section be?”

11. Follow-up Questions

- “What does Amdahl’s law say about your lock?” “How do you find your hottest lock?” (perf lock, mutex profilers, JFR)

12. Coding/Debugging Scenarios

- p99 spikes traced to a lock held across a network call → move the call out, hold the lock only to swap results in.

13. Best Practices

One lock per invariant; document what each lock guards; no I/O under locks; prefer data partitioning (sharded locks) over one hot lock.

14. Practice Questions

1. Rewrite: `lock { row = db.read(k); row.v++; db.write(row); }` to minimize the CS.
2. Which of mutual exclusion / progress / bounded waiting does a plain spinlock violate?

3.3 Mutex

1. Why Interviewers Ask This

“How does a mutex actually work?” is the definitive senior filter — they want futex mechanics, not “it’s a lock”.

2. Core Concept

A mutex (mutual exclusion lock) grants exclusive ownership: acquire blocks until available; only the owner may release. Sleeps the waiter rather than spinning (vs spinlock), has an owner (vs binary semaphore).

3. Internal Working

Modern Linux mutexes (pthread) are built on **futex** (“fast userspace mutex”): - **Fast path** (uncontended): one atomic CAS on a user-space word (0 → 1 locked). No syscall. ~15–25 ns. - **Slow path** (contended): set state to “locked with

waiters” (e.g., 2), call `futex(FUTEX_WAIT)` — kernel sleeps the thread on a wait queue keyed by that memory address. - **Unlock**: atomic set 0; if waiters flag was set, `futex(FUTEX_WAKE)` one thread. - Many implementations spin briefly before sleeping (adaptive mutex) to win when hold times are tiny. Variants: recursive (re-entrant), timed (`trylock/timeout`), PI (priority inheritance), robust (owner died).

4. ASCII Diagram

```
lock():
  CAS(state, 0 -> 1) success? ----> in critical section (NO syscall)
    | fail
    set state=2; futex_wait(&state,2) -> kernel sleep
unlock():
  state=0; if had waiters: futex_wake(&state,1)
Uncontended cost ~ one atomic op; contended cost ~ 2 syscalls + ctx switch (~ us)
```

5. Real Production Example

Every language runtime: Java `synchronized` (biased → thin → fat lock inflation, fat = OS mutex), Go `sync.Mutex` (CAS + semaphore/futex, with a “starvation mode” for fairness after 1 ms of waiting), glibc `pthread_mutex_t`.

6. Advantages

Simple ownership semantics; blocked waiters consume no CPU; ubiquitous and well-optimized; PI/robust variants for hard cases.

7. Trade-offs

Contended path costs syscalls + context switches (μ s); unfair by default (barging/lock convoys); can deadlock; priority inversion without PI.

8. Common Mistakes

- Saying “mutex always makes a syscall” — the uncontended fast path doesn’t (that’s the whole point of futexes).
- Unlocking from a different thread (UB for mutexes; legal for semaphores).
- Using recursive mutexes to paper over unclear ownership.

9. Performance Implications

Uncontended: nanoseconds. Contended: microseconds each (wait + wake + switch) plus cache-line transfer of the lock word between cores. Throughput of a hot mutex collapses past ~50–70% utilization — shard or redesign before that.

10. Common Interview Questions

- “Explain how a futex-based mutex works, fast and slow path.”
- “Mutex vs spinlock — when each?” “Mutex vs binary semaphore?”

11. Follow-up Questions

- “What is an adaptive mutex?” “What is lock convoy/barging?” “What happens if the owner dies?” (robust mutex, EOWNERDEAD)

12. Coding/Debugging Scenarios

- `perf` shows heavy `futex` syscall time → contended mutex; find it (`perf lock record`, JFR monitor-blocked events), then shrink CS/shard.
- Java service: profiler shows lock inflation on one `synchronized` map → replace with `ConcurrentHashMap` or striped locks.

13. Best Practices

Scope-based lock guards (RAII/defer/try-with); consistent acquisition order; prefer trylock+timeout on cross-subsystem locks; keep the lock word on its own cache line if adjacent hot data (false sharing).

14. Practice Questions

1. Implement a mutex on top of `futex_wait/futex_wake` with the 0/1/2 state machine — why is state 2 needed? (avoid lost wakeups / needless FUTEX_WAKE)
2. Go's mutex "starvation mode": what problem does it fix and at what cost?

3.4 Semaphore

1. Why Interviewers Ask This

Semaphores test whether you understand *counting resources* and signaling between threads — plus the classic producer-consumer implementation.

2. Core Concept

A semaphore is a counter with two atomic ops: `wait/P/acquire` (decrement; block if would go below 0) and `signal/V/release` (increment; wake a waiter). **Counting** semaphore = N permits (resource pool); **binary** = 0/1 (mutex-like, but no ownership — any thread may signal).

3. Internal Working

Kernel/runtime keeps `{count, wait queue}`; wait: `count--; if (count<0) sleep(queue)`; signal: `count++; if (count<=0) wake_one(queue)`. On Linux built on futexes (`sem_t`), in Java on AQS (`java.util.concurrent.Semaphore`).

4. ASCII Diagram

```
Bounded buffer (size N) with 3 semaphores:
empty = N, full = 0, mutex = 1

Producer:          Consumer:
wait(empty)         wait(full)
wait(mutex)         wait(mutex)
  buffer.put(item)    item = buffer.take()
signal(mutex)       signal(mutex)
signal(full)        signal(empty)
ORDER MATTERS: wait(mutex) before wait(empty) => deadlock!
```

5. Real Production Example

- Connection pools (DB pool of 50 = `semaphore(50)`).
- Rate limiting concurrent requests to a fragile downstream (bulkhead pattern, e.g., Semaphore isolation in resilience libraries).
- Bounded job queues; JDK `Semaphore` guarding native resource handles.

6. Advantages

Counts resources naturally; works across processes (named/POSIX semaphores); signaler needn't be the "owner" → usable for ordering/notification between different threads.

7. Trade-offs

No ownership → can't detect misuse (double signal silently raises capacity), no priority inheritance; easy to deadlock with wrong wait ordering; "who signals whom" logic is implicit and fragile vs condition variables.

8. Common Mistakes

- Swapping `wait(empty)` / `wait(mutex)` order in producer-consumer (deadlock — memorize why).
- Using a binary semaphore as a mutex and claiming they're identical (ownership, PI, error checking differ).
- Forgetting a `signal` on an error path (permit leak → pool drains to zero over days).

9. Performance Implications

Same futex cost model as mutex. A semaphore-guarded pool serializes on the semaphore's cache line at high rates — for very hot paths, use per-CPU/sharded pools.

10. Common Interview Questions

- “Implement producer-consumer with semaphores.” (top-5 classic — write it cold)
- “Binary semaphore vs mutex?” “How would you implement a semaphore with a mutex + condition variable?”

11. Follow-up Questions

- “What happens with 2 producers and 2 consumers — does your code still work?” (yes, mutex protects the buffer)
- “How do you implement readers-writers with semaphores?” “What if a consumer crashes holding the mutex?” (robust/futex cleanup, or don't share across processes casually)

12. Coding/Debugging Scenarios

- Service gradually loses DB pool capacity → permit leak on exception path; fix with try/finally release.
- Throughput capped exactly at N → semaphore bound is the bottleneck; resize or shard.

13. Best Practices

Always release in `finally`; name semaphores for the resource they count; prefer higher-level bounded queues/pools from your language's stdlib.

14. Practice Questions

1. Write producer-consumer for buffer size 1 — which semaphore values change?
2. Implement `Semaphore` using one mutex + one condition variable + int.
3. Use a semaphore to limit a crawler to 100 concurrent fetches — sketch code.

3.5 Spinlock

1. Why Interviewers Ask This

Spin vs sleep is a fundamental cost trade-off; kernel code and low-latency systems live on it. Interviewers probe whether you know *when spinning wins*.

2. Core Concept

A spinlock busy-waits (loops on an atomic test) instead of sleeping. Wins when expected hold time < ~2 context switches (a few μ s); mandatory in contexts that cannot sleep (interrupt handlers, holding a raw kernel spinlock).

3. Internal Working

- Naive: loop on `CAS(lock, 0->1)` — hammers the cache line with exclusive ownership requests (bus traffic).
- **Test-and-test-and-set (TTAS)**: spin on a *read* (line stays Shared) and only CAS when observed free — far less coherence traffic.
- Add exponential backoff and `PAUSE` instruction (SMT-friendly, saves power).
- **Ticket lock**: `my = fetch_add(next_ticket); while (serving != my) spin;` — FIFO fair, but all spinners watch one word.
- **MCS/qspinlock**: each waiter spins on its *own* cache line in a queue node — scalable; Linux uses qspinlocks.
- Kernel rule: spinlock holders disable preemption (and IRQ variants disable interrupts) — never sleep while holding one.

4. ASCII Diagram

```
TAS spinner: while (xchg(&l,1)) ;           <- line ping-pongs (bad)
TTAS spinner: while (l==1) pause;          <- spin on local cached read
              until CAS(&l,0,1) succeeds
MCS: [holder] -> [w1 spins on w1.flag] -> [w2 spins on w2.flag]
              unlock passes the lock to w1 by writing w1.flag (one line touched)
```

5. Real Production Example

Linux kernel spinlocks protect run queues, drivers, network stack fast paths; DPDK/HFT user-space spinlocks for sub- μ s critical sections; adaptive mutexes spin-then-sleep (glibc, Java, Go all do bounded spinning first).

6. Advantages

No syscall/context switch; latency of handoff $\sim$ nanoseconds; usable where sleeping is illegal.

7. Trade-offs

Burns CPU while waiting; disastrous if the holder is preempted (waiters spin a full time slice — why user-space pure spinlocks are dangerous); fairness and coherence-traffic issues unless queue-based.

8. Common Mistakes

- Using spinlocks in user space with more runnable threads than cores.
- Not knowing TTAS vs TAS or why MCS exists (senior-level differentiator).
- Sleeping/allocating (which may sleep) while holding a kernel spinlock.

9. Performance Implications

Break-even: spin if hold time $< \sim 1\text{--}2\times$ context-switch cost. Under oversubscription, spinlocks invert: throughput collapses as spinners waste the holder's CPU (lock-holder preemption; also hurts VMs — hence paravirtual spinlocks).

10. Common Interview Questions

- “Spinlock vs mutex — when each?” “Implement a spinlock; now make it scalable.”

11. Follow-up Questions

- “Why does the **PAUSE** instruction exist?” “What is lock-holder preemption in VMs?” “Why does Linux use queued spinlocks?”

12. Coding/Debugging Scenarios

- 100% CPU but low throughput; perf shows cycles in lock loop $\rightarrow$ oversubscribed spinlock; switch to adaptive mutex or reduce thread count to core count.

13. Best Practices

Default to adaptive mutexes; pure spinlocks only for pinned-thread, cannot-sleep, sub- μ s sections; bound the spin then fall back to sleeping.

14. Practice Questions

1. Implement TAS, TTAS, and ticket locks; explain the coherence traffic of each.
2. Your VM guest's spinlock throughput drops $50\times$ under host CPU contention — explain.

3.6 Read-Write Lock

1. Why Interviewers Ask This

Tests whether you can exploit workload asymmetry (read-mostly) and whether you know the hidden costs: writer starvation and the shared reader counter.

2. Core Concept

Allows either **many concurrent readers** or **one exclusive writer**. Ideal for read-mostly data (config, routing tables, caches).

3. Internal Working

State $\approx$ {reader count, writer flag, wait queues}. Policies: - **Reader-preference**: readers enter if no active writer $\rightarrow$ continuous readers **starve writers**. - **Writer-preference**: arriving writer blocks new readers $\rightarrow$ readers can starve under write bursts. - **Fair/FIFO**: grant in arrival order (phase-fair). pthread default is implementation-defined; Java `ReentrantReadWriteLock(fair=true)` optional; Go `sync.RWMutex` blocks new readers once a writer waits. Every reader acquire/release atomically updates a shared counter $\rightarrow$ the counter's cache line bounces across cores, so at high core counts read-lock overhead can exceed the read itself.

4. ASCII Diagram

```
Readers: R1 ===  R2 ===  R3 ===      (concurrent, overlapping)
Writer:  W ..... (waits for all readers, then exclusive)
Reader-pref: R4,R5,R6 keep arriving -> W waits forever (starvation)
Writer-pref: once W queued, new readers queue behind W
```

5. Real Production Example

In-memory config/feature-flag store read on every request, updated every few minutes; kernel `rwsem`s; DB shared/exclusive latches on B-tree pages; RCU (below) as the kernel's answer when even reader counting is too slow.

6. Advantages

Read parallelism for read-mostly workloads; exclusive writes keep invariants simple.

7. Trade-offs

Heavier than a mutex (two queues, counter); writer or reader starvation depending on policy; reader-counter cache-line contention; upgrade (read $\rightarrow$ write) is deadlock-prone (two upgraders wait on each other) and usually banned.

8. Common Mistakes

- Using RW locks for write-heavy or tiny critical sections (a plain mutex is faster).
- Not knowing the starvation policy of your platform's lock.
- Attempting lock upgrade.

9. Performance Implications

Rule of thumb: wins when reads $\geq$ ~90% *and* the read section is long enough to amortize the heavier acquire. For very hot read paths, prefer RCU / epoch-based reclamation / immutable-snapshot-swap (readers pay ~0).

10. Common Interview Questions

- "Readers-writers problem: implement with semaphores or CV; discuss writer starvation."
- "When is an RW lock *slower* than a mutex?"

11. Follow-up Questions

- "What is RCU / how do you get zero-cost readers?" (publish new version via atomic pointer swap; reclaim old after grace period)
- "Why is read $\rightarrow$ write upgrade dangerous?"

12. Coding/Debugging Scenarios

- Config updates take minutes to apply under load → writer starvation with reader-preferring lock; switch policy or snapshot-swap.
- Profiling shows `rwlock_rdlock` hot with 64 cores → counter contention; move to RCU/immutable snapshot.

13. Best Practices

Read-mostly + long reads → RW lock; read-mostly + short reads → immutable snapshot + atomic pointer; write-mixed → mutex; document the starvation policy you chose.

14. Practice Questions

1. Implement first-readers-writers (reader pref) with semaphores; then fix writer starvation.
2. Design the flag store: 1M reads/s, 1 write/min — which mechanism and why?

3.7 Condition Variable

1. Why Interviewers Ask This

CVs expose the two subtlest concurrency rules — the *predicate loop* and *spurious wakeups*. Getting `while` vs `if` wrong is an instant signal.

2. Core Concept

A condition variable lets threads sleep until a **condition over shared state** becomes true. Always used with a mutex: `wait(cv, m)` atomically releases `m` and sleeps; on wakeup it re-acquires `m`. `signal` wakes one waiter; `broadcast` wakes all.

3. Internal Working

- `wait`: enqueue self on cv's queue **and** release mutex atomically (otherwise a signal between “release” and “sleep” is lost — the **lost wakeup** problem); sleep (futex); on wake, re-acquire mutex, return.
- **Mesa semantics** (all real systems): signal only *hints*; the condition may be false again by the time the waiter runs → must recheck: `while (!pred) wait(cv, m);`
- **Spurious wakeups**: the OS may wake you with no signal at all (permitted by POSIX) — same `while` loop handles it.
- Signal with the mutex held (safest); broadcast when different waiters wait for different predicates or when state change may satisfy many.

4. ASCII Diagram

```
Consumer:                Producer:
lock(m)                  lock(m)
while (queue.empty())    queue.push(x)
    wait(cv, m) <--- atomically signal(cv)
    item = queue.pop()    releases m, unlock(m)
    unlock(m)             sleeps,
                           re-acquires m on wake
WHY while? Mesa semantics + spurious wakeups + N waiters racing for 1 item.
```

5. Real Production Example

Every thread pool's task queue (`java.util.concurrent Condition`, Go channels internally, C++ `std::condition_variable`); bounded buffers; “wait until shutdown complete” barriers.

6. Advantages

Expresses “sleep until arbitrary predicate” cleanly; no busy waiting; composable with one mutex protecting the state.

7. Trade-offs

Easy to misuse (if-instead-of-while, signaling without the lock, wrong cv/mutex pairing); broadcast causes thundering herd (all wake, one wins, rest re-sleep).

8. Common Mistakes

- `if (!pred) wait();` — the classic.
- Calling `wait` without holding the mutex (UB) or signaling *before* any waiter and expecting it to be remembered (CVs are memoryless — unlike semaphores).
- Using one CV for two different predicates then `signal` waking the wrong class of waiter (use two CVs, e.g., `notFull/notEmpty`).

9. Performance Implications

Broadcast to N waiters = N wakeups, N mutex re-acquisitions serialized → herd. Prefer `signal` when any single waiter can make progress; two CVs for producer-consumer avoid waking producers for consumer events.

10. Common Interview Questions

- “Why must condition-variable wait be in a while loop?” (Mesa + spurious + multiple waiters)
- “Implement a bounded blocking queue with mutex + 2 CVs.” (top-5 coding classic)

11. Follow-up Questions

- “Mesa vs Hoare semantics?” (Hoare: signal transfers ownership immediately — textbook only)
- “Why must release-and-sleep be atomic?” (lost wakeup)
- “CV vs semaphore for signaling?” (semaphore remembers signals; CV needs state under a mutex)

12. Coding/Debugging Scenarios

- Rare hang: consumer waits forever though queue non-empty → producer signaled before consumer waited *and* consumer checked with `if` under a race, or signaled without lock; fix pattern.
- CPU spike at every enqueue → broadcast thundering herd; switch to signal.

13. Best Practices

Always: hold mutex, `while` predicate, prefer signal over broadcast, one CV per predicate, state changes before signal.

14. Practice Questions

1. Bounded blocking queue (put/take, capacity N) — write it in your language, cold.
2. Implement `CountDownLatch` with mutex + CV.
3. What breaks if `wait` released the mutex *then* enqueued itself?

3.8 Monitor

1. Why Interviewers Ask This

Mostly via Java: `synchronized/wait/notify` is a monitor. Tests whether you can name the pattern and its encapsulation benefit.

2. Core Concept

A monitor = an object bundling **data + mutex + condition variable(s)**, where all public methods run under the mutex. Mutual exclusion is implicit and encapsulated, unlike raw mutexes sprinkled through code.

3. Internal Working

Java: every object has a monitor — `synchronized` acquires it; `wait()`/`notify()`/`notifyAll()` are the built-in single condition queue. HotSpot lock inflation: thin lock (CAS on header word) → fat lock (OS mutex + queues) under contention. C#'s `lock`, Python's condition objects, and “synchronized classes” follow the same pattern. Limitation: one implicit CV per object → use explicit `Lock`+multiple `Condition`s when you need notFull/notEmpty separation.

4. ASCII Diagram

```
+----- Monitor object -----+
| private state (queue, count, ...) |
| implicit mutex (entry set)        |
| condition queue(s) (wait set)     |
| synchronized put(x){ while(full) wait(); |
|     ...; notifyAll(); }            |
+-----+
Threads: [entry set: want lock] [wait set: called wait()]
```

5. Real Production Example

Any pre-java.util.concurrent Java code; `Hashtable`/`Vector` (fully synchronized monitors — and why they don't scale); guarded objects in Kotlin/C# services.

6. Advantages

Encapsulation: impossible to touch state without the lock; simpler mental model; language support (compiler-enforced release even on exception).

7. Trade-offs

One-lock-per-object coarseness; Java's single wait set forces `notifyAll` in mixed-waiter designs; monitor-based collections serialize everything (vs `ConcurrentHashMap` striping/CAS).

8. Common Mistakes

- `notify()` when waiters wait on different conditions (lost signal to the wrong waiter → hang) — use `notifyAll` or explicit Conditions.
- Synchronizing on a public object/string literal (external code can lock it → accidental contention/deadlock).
- Believing monitors prevent deadlock (nested monitor calls deadlock like any nested locks).

9. Performance Implications

Java uncontended `synchronized` is nearly free (thin lock CAS); contended → inflation to OS mutex. Monitor-per-collection designs cap throughput — measure and move to concurrent structures.

10. Common Interview Questions

- “How does `synchronized` work internally?” (monitor, lock inflation)
- “`wait/notify` vs `Lock/Condition`?” “Why `notifyAll` usually?”

11. Follow-up Questions

- “What happens if you call `wait()` outside `synchronized`?” (`IllegalMonitorStateException`)
- “Monitor vs semaphore as a language design choice?”

12. Coding/Debugging Scenarios

- Thread dump shows threads in `Object.wait()` forever → a `notify` chose the wrong waiter class; change to `notifyAll` or split conditions.

13. Best Practices

Lock on private final objects; keep synchronized methods short; prefer `java.util.concurrent` structures over hand-rolled monitors.

14. Practice Questions

1. Implement the bounded buffer as a Java monitor with `wait/notifyAll`; then with `ReentrantLock` + two `Conditions` — compare.
2. Why is `Vector` “thread-safe” yet `for(i<v.size()) v.get(i)` still racy? (composite operation — check-then-act across method calls)

3.9 Atomic Operations

1. Why Interviewers Ask This

Foundation of lock-free code and of every lock’s implementation. Senior interviews probe memory ordering, not just “it’s indivisible”.

2. Core Concept

An atomic operation completes as one indivisible unit — no thread observes it half-done. Hardware provides atomic load/store (aligned word), exchange, fetch-and-add, compare-and-swap. Atomics also carry **memory-ordering** semantics (acquire/release/seq_cst) that constrain how surrounding reads/writes may be reordered.

3. Internal Working

- x86: `LOCK`-prefixed instructions (e.g., `lock xadd`, `lock cmpxchg`) — the core holds the cache line in **Modified** state exclusively for the duration (modern CPUs lock the line, not the bus).
- ARM: LL/SC (`ldxr/stxr`) retry loops, or newer single-instruction atomics (LSE).
- Cost: uncontended atomic ≈ 10–20 cycles more than a plain op; **contended** atomics serialize on cache-line ownership — every op transfers the line between cores (~40–100+ ns each).
- Ordering: **acquire** (later ops can’t move before it — for lock acquire/reads of a flag), **release** (earlier ops can’t move after — for publish/unlock), **seq_cst** (single global order, priciest), **relaxed** (atomicity only — counters). Java `volatile` ≈ seq_cst load/store; Go’s `sync/atomic` is sequentially consistent.

4. ASCII Diagram

```

Publish pattern (why ordering matters):
T1: data = 42;           T2: while(!flag.load(acquire));
    flag.store(1, release);   use(data); // guaranteed 42
Without release/acquire, CPU/compiler may reorder:
    flag=1 visible BEFORE data=42 -> T2 reads garbage.
  
```

5. Real Production Example

Metrics counters (`fetch_add` relaxed), sequence numbers, ring buffers (Disruptor), refcounts (`shared_ptr`, kernel `atomic_t`), once-init flags, lock implementations themselves.

6. Advantages

No blocking, no deadlock, no priority inversion; nanosecond-scale; signal-handler and interrupt safe (locks are not).

7. Trade-offs

Only single-word granularity — multi-field invariants still need locks or careful protocols; memory-ordering bugs are the hardest class to debug; contended atomics still serialize (they’re not free parallelism).

8. Common Mistakes

- “Atomic = fast under contention” — a contended atomic counter can bottleneck a 64-core box (fix: per-CPU/sharded counters, e.g., `LongAdder`).
- Assuming two separate atomics update atomically *together*.
- Using relaxed ordering for a publish/subscribe flag.

9. Performance Implications

Uncontended: ~ns. Contended: cache-line ping-pong, ~100 ns+/op and falling throughput with more cores (negative scaling). Also **false sharing**: two unrelated atomics on one 64-byte line contend — pad/align hot fields.

10. Common Interview Questions

- “How is `fetch_add` implemented in hardware?” “What does `volatile` do in Java vs C?”
- “Why do we need memory barriers at all?” (compiler + CPU reordering, store buffers)

11. Follow-up Questions

- “Acquire/release vs `seq_cst` — give a use for each.” “What is false sharing and how do you fix it?”
- “Why is a global atomic counter a bad metrics design at 64 cores?” (per-CPU shards, sum on read)

12. Coding/Debugging Scenarios

- Throughput *drops* when adding cores; perf shows one hot cache line → contended atomic/false sharing; shard or pad.
- ARM-only crash of a lock-free queue that “worked on x86” → missing barriers (x86’s stronger ordering hid the bug).

13. Best Practices

Default `seq_cst` until profiling says otherwise; encapsulate lock-free code behind tested abstractions; pad contended fields to cache lines; prefer `stdlib` concurrent primitives.

14. Practice Questions

1. Implement a spinlock with `atomic_exchange` + acquire/release — annotate each ordering choice.
2. Why can’t you implement a correct mutex with plain loads/stores on modern hardware?
3. Design a 64-core-friendly request counter.

3.10 Compare-And-Swap (CAS)

1. Why Interviewers Ask This

CAS is *the* lock-free primitive; interviews test the retry-loop pattern and the ABA problem — a sharp senior discriminator.

2. Core Concept

`CAS(addr, expected, desired)`: atomically, if `*addr == expected` then `*addr = desired`, return success; else return failure (and the current value). Universal building block: locks, lock-free stacks/queues, optimistic updates.

3. Internal Working

- Hardware: x86 `lock cmpxchg`; ARM LL/SC pair.
- **Retry loop** (optimistic concurrency):

```
do { old = load(addr); new = f(old); } while (!CAS(addr, old, new));
```

- **Lock-free** guarantee: some thread always makes progress (a failed CAS means *another* CAS succeeded) — but any individual thread may starve (not wait-free).
- **ABA problem**: value went $A \rightarrow B \rightarrow A$ between your load and CAS; CAS sees “A” and succeeds, but the *structure* changed (e.g., stack node freed and reallocated at same address → corruption). Fixes: version-tagged pointers (double-width CAS), hazard pointers/epoch reclamation, or GC languages (a “same” reference really is the same object).

4. ASCII Diagram

```

Lock-free stack push:
do {
    node->next = top;
} while(!CAS(&top, node->next,
            node));

ABA on pop:
T1: reads top=A, next=B ... (paused)
T2: pop A, pop B, push A (A reused!)
T1: CAS(&top, A, B) SUCCEEDS
    -> top=B, but B was freed! CORRUPT
Fix: CAS on (pointer, version) pair

```

5. Real Production Example

`java.util.concurrent` everywhere (AtomicLong, ConcurrentHashMap bins, AQS state); Go runtime scheduler queues; kernel lockless fast paths; **optimistic locking in databases** (`UPDATE ... WHERE version = ?`) — CAS at the row level; etcd/ZooKeeper compare-and-set on version = distributed CAS.

6. Advantages

Non-blocking: no deadlock, no priority inversion, no lock-holder-preemption; great under low/moderate contention; composable into optimistic patterns at every layer (memory → DB → distributed).

7. Trade-offs

Retry storms under high contention (wasted work, line ping-pong); single-word scope; ABA; algorithms are notoriously hard to prove correct (use libraries).

8. Common Mistakes

- Recomputing an expensive `f(oid)` in a hot retry loop.
- Ignoring ABA in manual-memory-management languages.
- Claiming lock-free = wait-free (individual starvation still possible).

9. Performance Implications

Low contention: CAS ≈ one atomic (~ns) — beats mutex. High contention: failure rate soars; `fetch_add` (always succeeds) beats CAS-loop for counters; backoff or sharding needed. Rule: CAS for rarely-contended state; locks for complex/contended invariants.

10. Common Interview Questions

- “Implement `AtomicInteger.incrementAndGet` with CAS.” “Explain the ABA problem with a concrete corruption scenario.”
- “How does `ConcurrentHashMap.putIfAbsent` relate to CAS?”

11. Follow-up Questions

- “Lock-free vs wait-free?” “How does LL/SC differ from CAS regarding ABA?” (LL/SC fails on *any* write to the line → naturally ABA-immune, but can livelock)
- “How is DB optimistic locking the same idea?” (read version → attempt conditional write → retry on conflict)

12. Coding/Debugging Scenarios

- Heavy retry loop visible as high CPU with flat throughput → contention collapse; add exponential backoff or switch to a lock/sharding.
- Rare crash in a custom lock-free C++ queue only under load → ABA/reclamation bug; adopt hazard pointers or a proven library.

13. Best Practices

Prefer stdlib lock-free structures; keep `f(oid)` cheap and side-effect-free; bounded retries with backoff; in C/C++, pair CAS with a safe memory-reclamation scheme.

14. Practice Questions

1. Write a lock-free stack (push/pop) and point at the exact ABA line.

2. Implement optimistic row update with a version column; what's the retry policy?
3. When would you deliberately choose a mutex over CAS even at low contention? (multi-field invariant, need fairness/PI)

Module 3 Cheat Sheet (one page)

Primitive	Blocking?	Ownership	Best for	Killer detail
Mutex (futex)	Sleeps	Yes	General mutual exclusion	Uncontended = 1 CAS, no syscall
Spinlock	Busy-waits	Yes	<μs sections, no-sleep contexts	TTAS/MCS reduce cache traffic
Semaphore	Sleeps	No	Counting resources, pools	Any thread may signal; permits leak
RW lock	Sleeps	Yes	Read-mostly, longer reads	Writer starvation; reader-counter contention
Condition var	Sleeps	With mutex	Wait-for-predicate	<code>while</code> loop! Mesa + spurious wakeups
Monitor	Sleeps	Implicit	Encapsulated shared state	Java <code>synchronized</code> ; 1 wait set/object
Atomics	No	—	Counters, flags, publish	Ordering: acquire/release/seq_cst
CAS	No	—	Lock-free, optimistic	ABA; retry loops; lock-free ≠ wait-free

Costs: uncontended atomic/mutex ~ns; contended mutex ~μs (futex syscalls + switch); contended cache line ~100 ns/op and negative scaling. **Golden rules:** lock data not code; no I/O under locks; `while(!pred) wait()`; consistent lock order; shard hot state; prefer stdlib. **Producer-consumer:** `empty=N, full=0, mutex=1`; wait resource-semaphore *before* mutex.

Top Interview Questions

1. Why is `count++` racy — show instruction interleaving.
2. How does a mutex work internally (futex fast/slow path)?
3. Mutex vs spinlock vs semaphore — decision table with costs.
4. Implement producer-consumer (semaphores) and bounded blocking queue (mutex + 2 CVs).
5. Why `while` around `cv.wait()`?
6. Readers-writers with starvation discussion.
7. Explain CAS and the ABA problem with a corruption scenario.
8. False sharing and contended-atomic scaling collapse.

Common Mistakes (module-wide)

- `if` instead of `while` at CVs; signaling without the mutex.
- “Volatile makes ++ atomic”; “atomics are free”; “lock-free = wait-free”.
- Wrong semaphore ordering in producer-consumer (deadlock).
- I/O inside critical sections; lock upgrade attempts; ignoring writer starvation.
- User-space spinlocks with oversubscribed threads.

Mock Interview (self-test, ~25 min)

1. (Code) Bounded blocking queue with mutex + two condition variables — write, then defend every line.
2. (Depth) Walk through a futex mutex: two threads contend; list every atomic op, syscall, and state value.
3. (Design) A 64-core service updates one global stats counter 5M times/sec — it's the bottleneck. Fix it (sharded/per-CPU counters), estimate the win.

4. (Trap) “We replaced our mutex with an RW lock and got slower.” Give three plausible reasons.
5. (Trap) Your lock-free stack corrupts memory once a week in C++, never in the Java port. Explain (ABA + reclamation vs GC).
6. (Scenario) p99 latency spikes; `perf` shows 30% of cycles in `futex_wait`. Diagnose step by step.